Elementary Operations on Quasi-Toeplitz Matrices

PCCA6

Ali Arda BARUT
Rafael LLAMAS

May 26, 2026

Contents

Url of the project: https://github.com/Rafael-LLamas/Projet-CCA-groupe-6

1 Introduction

In computational numerical linear algebra, dense matrices often present a computational bottleneck, scaling quadratically in terms of memory usage ($O(nm)$) for a matrix with n rows and m columns and cubically in terms of time complexity ($O(n^3)$) for standard operations. However, in many real-world applications such as signal processing[3][7] and classical simulation of quantum systems[9], these dense matrices are not totally random. The elements are arranged in repeating, deterministic patterns. Designing algorithms that take advantage of this pattern can bypass the fundamental limitations of dense linear algebra.

To exploit these patterns, we apply a displacement operation to determine how much a matrix deviates from a perfect diagonal structure. For a highly structured matrix, this displacement results in a low rank. This demonstrates that it is possible to represent a dense matrix using two smaller, more compact matrices. Performing operations on these matrices reduces both storage and computation requirements. The aim of this project is to explore and implement elementary operations on quasi-Toeplitz matrices using these displacement operators.

1.1 Foundations of Toeplitz and quasi-Toeplitz Structures

In order to mathematically formalise the concepts introduced above, we must first define these structured patterns. This section outlines the properties of Toeplitz and quasi-Toeplitz matrices and introduces the displacement operators necessary for extracting their generators.

1.1.1 Toeplitz Matrices

A Toeplitz matrix is characterized by the property that all elements along any given diagonal are identical. For a matrix $A \in \mathbb{K}^{n\times m}$, it satisfies the condition $A_{i,j} = A_{i+1,j+1}$. In other words, a Toeplitz matrix is one in which each element in the first row and first column descends diagonally. It can be represented by a sequence of elements $\{a_k\}$ such that:

$$A = \begin{pmatrix} a_0 & a_{-1} & \cdots & & & & & a_{-(m-1)} \\ a_1 & a_0 & a_{-1} & & & & & a_{1-(m-1)} \\ \vdots & a_1 & a_0 & \ddots & & & & a_{2-(m-1)} \\ \vdots & & \ddots & \ddots & \ddots & & & \vdots \\ a_{n-1} & \cdots & \cdots & a_1 & a_0 & a_{-1} & \cdots & a_{(n-1)-(m-1)} \end{pmatrix}$$

$$A_{i,j} = a_{i-j}, \quad 0 \le i < n,\ 0 \le j < m$$

1.1.2 Displacement Operators

While a pure Toeplitz matrix is perfectly structured, real-world operations (such as matrix multiplication or inversion) often slightly corrupt this perfect diagonal pattern. Matrices that deviate slightly from a pure Toeplitz form are broadly referred to as quasi-Toeplitz or Toeplitz-like matrices.

Instead of defining these matrices through an additive correction term, we evaluate them operationally through their displacement. We define two complementary displacement operators:

$$\begin{aligned} \phi_+(A) &= A - ZAZ^T \\ \phi_-(A) &= A - Z^TAZ \end{aligned} \tag{1}$$

where Z represents the lower shift matrix (downshift) and Z^T represents the upper shift matrix (upshift). Essentially, Z is a matrix with ones on the first sub-diagonal and zeros everywhere else:

$$Z = \begin{pmatrix} 0 & & & 0 \\ 1 & & & \\ & \ddots & & \\ & & 1 & 0 \end{pmatrix} \in \mathbb{K}^{n\times n} \quad Z^T = \begin{pmatrix} 0 & 1 & & \\ & & \ddots & \\ & & & 1 \\ 0 & & & 0 \end{pmatrix} \in \mathbb{K}^{m\times m}$$

The two operators differ in their direction: ϕ_+ operates via a downshift, while ϕ_- operates via an upshift. For a generic Toeplitz matrix of size $n \times m$, the $\phi_+(A)$ subtraction results in zeros everywhere except for the first row and the first column, while $\phi_-(A)$ results in zeros everywhere except the last row and column:

$$\phi_+(A) = \begin{pmatrix} a_0 & a_{-1} & a_{-2} & \cdots & a_{-(m-1)} \\ a_1 & 0 & 0 & \cdots & 0 \\ a_2 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & 0 & 0 & \cdots & 0 \end{pmatrix} \in \mathbb{K}^{n\times m}$$

While the definitions in Equation (1) suggest a matrix-on-matrix subtraction with significant overhead, the structured nature of A allows for a much more efficient computation. By leveraging the Toeplitz structure, both displacements can be computed via simple loops:

$$(\phi_+(A))_{i,j} = \begin{cases} A_{i,j} & \text{if } i = 0 \text{ or } j = 0 \\ A_{i,j} - A_{i-1,j-1} & \text{otherwise} \end{cases} \tag{2}$$

$$(\phi_-(A))_{i,j} = \begin{cases} A_{i,j} & \text{if } i = n-1 \text{ or } j = m-1 \\ A_{i,j} - A_{i+1,j+1} & \text{otherwise} \end{cases} \tag{3}$$

In practice, we enable optimization through pointer arithmetic. By iterating from the bottom-right index (n, m) toward the origin $(0, 0)$ for ϕ_+, and in increasing order for ϕ_-, the displacements can be calculated in-place, resulting in a strict complexity bound of $O(nm)$.

1.1.3 Generator Matrices and Reconstruction

While the displacement matrix $\phi_+(A)$ occupies $n \times m$ elements, it is filled with zeros. Our primary interest in this project lies in structured matrices A where the rank of this displacement matrix is smaller than the matrix dimensions ($\alpha \ll n$ and $\alpha \ll m$).

For a standard Toeplitz matrix, this rank is strictly bounded ($\alpha \leq 2$). This is what allows A to be efficiently compressed. Instead of storing $\mathcal{O}(nm)$ elements, the matrix can be represented via a pair of highly compact matrices known as generators, denoted as $G \in \mathbb{K}^{n\times\alpha}$ and $H \in \mathbb{K}^{m\times\alpha}$, which satisfy:

$$\phi_+(A) = GH^T \tag{4}$$

To reconstruct the uncompressed operator A directly from its generator columns g_k and h_k (for $k = 1, \ldots, \alpha$), we formalize the families of lower and upper triangular Toeplitz operators.

For a column vector $v = [v_0, v_1, \ldots, v_{n-1}]^T \in \mathbb{K}^n$, we define the lower triangular Toeplitz operator $L_n(v) \in \mathbb{K}^{n\times n}$ as the square matrix. Let $U_{n,m}(w) \in \mathbb{K}^{n\times m}$ be the rectangular upper triangular Toeplitz operator generated by a vector $w = [w_0, w_1, \ldots, w_{m-1}]^T \in \mathbb{K}^m$. This operator is formed by taking the first n rows of the fully square $m \times m$ upper triangular matrix $L_m(w)^T$:

$$L_n(v) = \begin{pmatrix} v_0 & 0 & 0 & \cdots & 0 \\ v_1 & v_0 & 0 & \cdots & 0 \\ v_2 & v_1 & v_0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ v_{n-1} & v_{n-2} & \cdots & v_1 & v_0 \end{pmatrix} \quad U_{n,m}(w) = \begin{pmatrix} w_0 & w_1 & w_2 & \cdots & \cdots & w_{m-1} \\ 0 & w_0 & w_1 & \cdots & \cdots & w_{m-2} \\ 0 & 0 & w_0 & \cdots & \cdots & w_{m-3} \\ \vdots & \vdots & \ddots & \ddots & & \vdots \\ 0 & 0 & \cdots & 0 & w_0 & \cdots \end{pmatrix}$$

By denoting the individual columns of the generators as $G = [g_1, \ldots, g_\alpha]$ and $H = [h_1, \ldots, h_\alpha]$, the displacement inversion theorem [1] establishes that the full matrix A can be synthesized as a sum of products of these triangular structures:

$$GH^T = \phi_+(A) \quad \iff \quad A = \sum_{k=1}^{\alpha} L_n(g_k)\, U_{n,m}(h_k) \tag{5}$$

2 Basic Operations

Arithmetic operations on matrices are fundamental to algorithmic efficiency. In this section, we see how to fully exploit the properties of Toeplitz-like matrices to significantly reduce their computational complexity bounds. First, we detail an optimized approach to matrix addition. Next, we analyze the low computational cost of matrix-vector multiplication. Finally, we conclude by demonstrating how to perform full matrix multiplication directly using displacement generators.

Throughout this section, we consider structured matrices A and B, represented by their displacement generators (T, U) and (G, H) respectively, in accordance with the properties defined previously.

2.1 Addition

2.1.1 Generator Addition Algorithm

Using displacement generators, the addition of two structured matrices can be performed efficiently by simply concatenating their respective generators. As established by the properties of displacement operators (Properties 10.7 and 10.8 in [1]), this approach reduces the computational complexity of matrix addition from $O(n^2)$ for a naive dense algorithm down to $O(\alpha n)$.

Algorithm 1 Generator Addition

Input:
- T, U: Displacement generators of structured matrix $A \in \mathbb{K}^{n \times m}$
- G, H: Displacement generators of structured matrix $B \in \mathbb{K}^{n \times m}$

Output: G_C, H_C: Displacement generators representing the sum $A + B$

```
function GR_MAT_ADD_GENERATORS(T, U, G, H)
    G_C ← [T | G]        ▷ Horizontal concatenation
    H_C ← [U | H]        ▷ Horizontal concatenation
    return G_C, H_C
end function
```

2.1.2 Generator Addition Example

To illustrate this, let A and B be two Toeplitz-like matrices defined over the finite field $\mathbb{Z}/13\mathbb{Z}$, and let C denote their naive sum ($C = A + B \pmod{13}$):

$$A = \begin{pmatrix} 0 & 7 & 10 \\ 5 & 0 & 7 \\ 0 & 5 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 9 & 12 & 4 \\ 0 & 9 & 12 \\ 3 & 0 & 9 \end{pmatrix}, \quad C = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix}$$

Suppose the displacement generators for A are given by (T, U) and for B by (G, H):

$$T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad U = \begin{pmatrix} 5 & 0 \\ 0 & 7 \\ 0 & 10 \end{pmatrix}, \quad G = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 9 & 1 \end{pmatrix}, \quad H = \begin{pmatrix} 9 & 0 \\ 12 & 9 \\ 4 & 3 \end{pmatrix}$$

By applying the generator addition, we compute the generators of the result, G_C and H_C, through ordered horizontal concatenation ($G_C = [T \mid G]$ and $H_C = [U \mid H]$):

$$G_C = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 9 & 1 \end{pmatrix}, \quad H_C = \begin{pmatrix} 5 & 0 & 9 & 0 \\ 0 & 7 & 12 & 9 \\ 0 & 10 & 4 & 3 \end{pmatrix}$$

Reconstructing the dense matrix D directly from these concatenated generators yields:

$$D = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix}$$

The reconstructed matrix D is equal to C, validating the generator addition.

2.1.3 Generator Addition Benchmark

The performance benchmarks for the generator addition algorithm highlight a reduction in execution time compared to standard dense matrix operations:

n	Generators (ms)	Flint Dense (ms)
128	0.003577	0.01373
1024	0.03205	2.52
4096	0.1148	29.18

Table 1: Average runtimes of matrix addition over 10 runs.

To verify the theoretical complexity of $\mathcal{O}(\alpha n)$, we analyze the growth rate of the execution time $T(n)$ as the matrix dimension n increases. Assuming α remains a small constant, a linear time complexity dictates that multiplying the input size by a scalar k should increase the runtime by approximately the same factor k.

When scaling the matrix size from $n = 128$ to $n = 1024$ (an $8\times$ increase), the measured runtime ratio is:

$$\frac{T(1024)}{T(128)} \approx 8.96$$

Similarly, scaling from $n = 1024$ to $n = 4096$ (a $4\times$ increase) yields a ratio of:

$$\frac{T(4096)}{T(1024)} \approx 3.58$$

These observed ratios align with the expected theoretical factors of 8 and 4. The slight deviations from perfect linear result can be attributed to the low overall computational cost of the operation, but overall these empirical results assure the expected $O(\alpha n)$ time complexity.

2.2 Multiplication

2.2.1 Matrix-Vector Generator Multiplication

To perform full matrix multiplication efficiently using displacement generators, we must first define a subroutine that multiplies a structured matrix directly by a dense vector (or a block of vectors). We aim to avoid the necessity of reconstructing the dense matrix from its generators. We define $V \in \mathbb{K}^{m\times p}$ where p represents a variable number of columns. We modify the reconstruction formula 5 to apply this multiplication.

$$C = \sum_{k=1}^{\alpha} L(g_k)\, U(h_k)\, V \tag{6}$$

We evaluate the reconstruction formula by mapping the columns of the displacement generators and the vector into polynomials which allows us to compute from the triangular Toeplitz operators entirely through polynomial multiplication.

Algorithm 2 Multiplication of displacement generators with a vector/matrix

Input:
- G, H: Displacement generators of structured matrix $A \in \mathbb{K}^{n\times m}$
- V: Dense vector or matrix $\in \mathbb{K}^{m\times p}$

Output: C: Resulting product matrix $\in \mathbb{K}^{n\times p}$

function GR_MAT_MUL_VECTOR(G, H, V)
  $n \leftarrow$ nbRows(G)
  $m \leftarrow$ nbRows(H)
  Initialize C as a zero matrix of size $n \times$ nbCols(V) ▷ Initialize accumulator matrix
  for $j \leftarrow 0$ **to** nbCols(V) $- 1$ **do**
    $P_v \leftarrow \sum_{r=0}^{m-1} V[r, j]x^r$ ▷ Map the j-th column of V to a polynomial
    for $k \leftarrow 0$ **to** nbCols(G) $- 1$ **do**
      $P_g \leftarrow \sum_{r=0}^{n-1} G[r, k]x^r$ ▷ Map the k-th column of generator G
      $P_h \leftarrow \sum_{r=0}^{m-1} H[r, k]x^r$ ▷ Map the k-th column of generator H
      $P_{h\_rev} \leftarrow$ Reverse(P_h, m) ▷ Reverse H to simulate upper Toeplitz operator
      $P_{tmp} \leftarrow P_{h\_rev}P_v$ ▷ Apply upper Toeplitz structure to the vector
      $P_{tmp\_shift} \leftarrow P_{tmp} \gg (m-1)$ ▷ Shift to extract valid convolution coefficients
      $P_{res} \leftarrow P_g P_{tmp\_shift}$ ▷ Apply lower Toeplitz structure
      for $i \leftarrow 0$ **to** Length(P_{res}) $- 1$ **do**
        $C[i, j] \leftarrow C[i, j] +$ Coefficient(P_{res}, i) ▷ Accumulate results into C
      end for
    end for
  end for
  return C
end function

2.2.2 Matrix-Matrix Generator Multiplication

Using displacement generators, the full multiplication of two structured matrices can be computed by extracting the boundary shifts and concatenating the resulting generators. For matrices $A \approx (T, U)$ and $B \approx (G, H)$, this requires computing specific intermediate sub-blocks $W = ZAZ^TG$ and $V = B^TU$, as well as extracting the final boundary columns a and b by multiplying these sub blocks with the basis vector of $e_{m-1}^{(m)} = (0, ...0, 1)^T$.

By performing these operations entirely in the compressed generator space, we reduce the computational complexity from the naive $O(n^3)$ bound down to $O(\alpha^2 M(n))$, where $M(n)$ represents the time required to multiply two polynomials of degree n.

Algorithm 3 Generator Multiplication

Input:

- T, U: Displacement generators of structured matrix $A \in \mathbb{K}^{n \times m}$
- G, H: Displacement generators of structured matrix $B \in \mathbb{K}^{m \times k}$

Output: G_C, H_C: Displacement generators representing the product $A \times B$

function GR_MAT_MUL_GENERATORS(T, U, G, H)
  $e_{m-1}^{(m)} \leftarrow (0, \ldots, 0, 1)^T$ ▷ Basis vector to extract the last column
  $W_{Temp} \leftarrow Z^T \times G$
  $W \leftarrow Z \times$ GR_MAT_MUL_VECTOR(T, U, W_{Temp}) ▷ Vector multiplication of FLINT[8]
  $V \leftarrow$ GR_MAT_MUL_VECTOR(H, G, U)
  $a \leftarrow Z \times$ GR_MAT_MUL_VECTOR(T, U, e_{n-1})
  $b \leftarrow -Z \times$ GR_MAT_MUL_VECTOR(H, G, e_{n-1})
  $G_C \leftarrow [T \mid W \mid a]$ ▷ Horizontal concatenation
  $H_C \leftarrow [V \mid H \mid b]$ ▷ Horizontal concatenation
  return G_C, H_C
end function

2.2.3 Generator Multiplication Example

Let A and B be two random structured matrices, and let C denote their product. Every element is in $\mathbb{Z}/_{13}\mathbb{Z}$.

$$A = \begin{pmatrix} 10 & 5 & 8 \\ 5 & 10 & 5 \\ 3 & 5 & 10 \end{pmatrix}, B = \begin{pmatrix} 0 & 3 & 0 \\ 3 & 0 & 3 \\ 0 & 3 & 0 \end{pmatrix}, C = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \tag{7}$$

We take displacement generators:

$$T = \begin{pmatrix} 1 & 0 \\ 7 & 1 \\ 12 & 11 \end{pmatrix}, U = \begin{pmatrix} 10 & 0 \\ 5 & 4 \\ 8 & 9 \end{pmatrix}, G = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, H = \begin{pmatrix} 3 & 0 \\ 0 & 3 \\ 0 & 0 \end{pmatrix} \tag{8}$$

We need to calculate W and V before concatenating the generators of A and B to obtain the displacement generator of D.

$$W = \begin{pmatrix} 0 & 0 \\ 10 & 0 \\ 5 & 0 \end{pmatrix}, V = \begin{pmatrix} 2 & 12 \\ 2 & 1 \\ 2 & 12 \end{pmatrix} \tag{9}$$

Then we do the concatenation of T,W,a and V,H,-b respectively to obtain RES_G and RES_H.

$$RES_G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 7 & 1 & 10 & 0 & 8 \\ 12 & 11 & 5 & 0 & 5 \end{pmatrix}, RES_H = \begin{pmatrix} 2 & 12 & 3 & 0 & 0 \\ 2 & 1 & 0 & 3 & 0 \\ 2 & 12 & 0 & 0 & 10 \end{pmatrix} \tag{10}$$

The resulting matrix D :

$$D = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \tag{11}$$

We can see that the matrix D is equal to C, our way to multiply the 2 matrices is correct.

2.2.4 Generator Multiplication Benchmark

The performance benchmarks for both the matrix-vector and full matrix-matrix generator multiplications show improvements over standard dense operations:

n	Generators (ms)	Flint Dense (ms)	Vector (ms)	Vector Flint (ms)
128	0.3493	2.461	0.03261	0.01987
1024	2.460	77.69	0.3166	1.017
4096	9.991	38150	1.054	18.41

Table 2: Average runtimes of multiplication operations over 10 runs of rank 2.

To validate the time complexity of the matrix-vector multiplication, we analyze the execution time. For an expected theoretical complexity of $O(\alpha\mathcal{M}(n))$ the expected scaling factors when increasing n from 128 to 1024 (8×) and from 1024 to 4096 (4×) are approximately 9.14 and 4.4 for $M(n) = O(nlogn)$, respectively:

$$\frac{T(1024)}{T(128)} \approx 9.71, \quad \frac{T(4096)}{T(1024)} \approx 3.33$$

While the first ratio aligns closely with theoretical expectations, the second shows some deviation. This can be explained to the overhead of getting matrix values into polynomial structures. This operation scales at $O(\alpha n)$, a cost that remains non-negligible compared to $M(n)$ in these dimensions. Tests with larger matrices confirm that the $O(\alpha M(n))$ complexity holds. Future optimizations to this polynomial copy would reduce the overhead.

For the full generator multiplication, the expected complexity is $(\alpha^2 M(n))$. The observed scaling ratios are:

$$\frac{T(1024)}{T(128)} \approx 7.06, \quad \frac{T(4096)}{T(1024)} \approx 4.06$$

Here, the second ratio closely approaches the expected 4.4, while the first ratio shows more deviation. Because the full multiplication repeatedly copies values into polynomials again in function call, it has the memory overhead to $O(4 \times \alpha \times n)$ for counting 4 calls of the function. This introduced a overhead over with smaller α.

α	$n = 256$ (ms)	$n = 512$ (ms)
2	0.7917	1.694
4	2.677	5.468
8	8.913	19.74
16	32.55	75.56
32	124	285.2

Table 3: Average runtimes of generator multiplicaiton scaling by rank over 10 runs

If we compare the result between rank 4 , 8 and 16 with the same techniques we use before but this time the expected ratio is 2.25 if $M(n) = nlogn$:

$$\frac{T_4(512)}{T_4(256)} \approx 2.04, \quad \frac{T_8(512)}{T_8(256)} \approx 2.21, \frac{T_{16}(512)}{T_{16}(256)} \approx 2.32$$

We see the ratio for $\alpha = 4$ is below what expected but for alpha = 8 and alpha = 16, the ratios are really close to 2.25, that means the complexity is correct for $\alpha \geq 8$. A future optimisation can reduce this overhead to make this function more beneficial for smaller ranks matrices. However, as n scales, polynomial arithmetic correctly dominates the runtime, respecting the bound of $O(\alpha^2 \mathcal{M}(n))$.

2.3 The Challenge of Rank Growth

While the arithmetic operations described above achieve efficient complexities, chaining these operations introduces a severe vulnerability. As demonstrated in the generator addition and multiplication algorithms, the output generators are formed via horizontal concatenation. Consequently, performing arithmetic on two matrices with a displacement rank of α inflates the resulting matrix's displacement rank (ex. addition results in a rank of 2α).

If multiple operations are performed sequentially such as in divide-and-conquer matrix inversion or iterative system solvers, this rank will grow exponentially. To maintain our efficiency across complex computations, we must actively reduce the dimensions of these intermediate generators without changing the source dense matrix they represent.

3 Generator Compression

As shown at the end of section 2, operations cause the displacement rank α to grow rapidly. To maintain efficiency, we must compress the generator matrices.

Let $G \in \mathbb{K}^{n \times \alpha}$ and $H \in \mathbb{K}^{n \times \alpha}$ be displacement generators representing a displacement matrix $\nabla A = GH^T$. The true mathematical rank of the displacement matrix is $r = \text{rank}(\nabla A) < \alpha$.

Goal: Compute new, compressed generators $G' \in \mathbb{K}^{n \times r}$ and $H' \in \mathbb{K}^{n \times r}$, where r is minimal and:

$$G'H'^T = GH^T$$

We achieve this reduction by sequentially compressing each matrix using an LU factorization and applying the inverse operations to the other matrix to preserve the product.

Algorithm 4 Generator Compression

Input:
- G_A, H_A: Uncompressed generators $\in \mathbb{K}^{n \times \alpha}$

Output: G_D, H_D: Compressed generators $\in \mathbb{K}^{n \times r}$ where $r \leq \alpha$

1: **function** GR_MAT_GENERATOR_COMPRESS(G_A, H_A)
2: $G_1, H_1 \leftarrow$ GR_MAT_GENERATOR_COMPRESS_AUX(G_A, H_A)
3: $H_D, G_D \leftarrow$ GR_MAT_GENERATOR_COMPRESS_AUX(H_1, G_1)
4: **return** G_D, H_D
5: **end function**

Algorithm 5 Generator Compression Auxiliary

Input:
- G, H: Generators to be partially compressed

Output: G_D, H_D: Partially compressed generators

function GR_MAT_GENERATOR_COMPRESS_AUX(G, H)
  $P, L, U \leftarrow$ LU_Decomposition(G^T) ▷ $PG^T = LU$
  $r \leftarrow$ Number of non-zero rows in U ▷ Extracts the true rank r
  if $r < 1$ **then**
    return Generators of a 0 matrix
  end if
  $G_D \leftarrow (U_{[0:r-1],*})^T$
  $H_D \leftarrow HP^T L_{*,[0:r-1]}$
  return G_D, H_D
end function

Note that calling the auxiliary function twice with swapped arguments is valid because each individual call inherently preserves the matrix product.

Proposition 1. *Algorithm 4 is correct. Specifically, the auxiliary function preserves the generator product, guaranteeing that $H_D G_D^T = H_1 G_1^T$, which consequently ensures $G_D H_D^T = G_A H_A^T$.*

Proof. We show that the second call to the auxiliary function preserves the generator product. In this step, the first argument is H_1, meaning the LU decomposition is performed on H_1^T, resulting $PH_1^T = LU$.

Since U has a rank of r, the matrix product depends exclusively on the first r columns and rows:

$$LU = L_{*,[0:r-1]}\, U_{[0:r-1],*}$$

Starting from $PH_1^T = LU$ and multiplying both sides on the left by P^T, we obtain:

$$\begin{aligned} H_1^T &= P^T L_{*,[0:r-1]}\, U_{[0:r-1],*} \\ H_1 &= \left(U_{[0:r-1],*}\right)^T L_{*,[0:r-1]}^T\, P \end{aligned}$$

Based on the logic of the auxiliary function, the returned generators are $H_D = \left(U_{[0:r-1],*}\right)^T$ and $G_D = G_1 P^T L_{*,[0:r-1]}$. Taking the transpose of G_D gives:

$$G_D^T = \left(G_1 P^T L_{*,[0:r-1]}\right)^T = L_{*,[0:r-1]}^T\, P\, G_1^T$$

Substituting these into the product $H_D G_D^T$, we reconstruct the original form of H_1:

$$\begin{aligned} H_D G_D^T &= \left(U_{[0:r-1],*}\right)^T \left(L_{*,[0:r-1]}^T\, P\, G_1^T\right) \\ &= \left(\left(U_{[0:r-1],*}\right)^T L_{*,[0:r-1]}^T\, P\right) G_1^T \\ &= H_1 G_1^T \end{aligned}$$

Since $H_D G_D^T = H_1 G_1^T$, taking the transpose of both sides gives us $G_D H_D^T = G_1 H_1^T$. □

Computing the LU factorization of the transposed generator $G^T \in \mathbb{K}^{\alpha \times n}$ requires $O(\alpha^2 n)$ operations. Extracting U and applying the permutation matrix P^T to H is in $O(\alpha n)$[1]. Computing

[1] In FLINT[8], the permutation matrix is represented as an array of indices. We do a swapping operation rather than a full matrix multiplication.

$H_D = HP^T \times L_{*,[0:r-1]}$ contains a multiplication of a $n \times \alpha$ matrix by a $\alpha \times r$ matrix. Using standard matrix multiplication, this takes $O(n\alpha r)$ operations. Our implementation uses loops to extract the selected amount of rows and columns, but their complexities are strictly bounded by $O(n\alpha r)$.

We can generalize this complexity: for any generator matrices in $\mathbb{K}^{n\times m}$ compressing to rank r, the overall time complexity for the generator compression is dominated by the matrix multiplication and is bounded by $O(rnm)$.

4 Generator Inversion

4.1 Algorithm

We present an implementation of Strassen's inversion method specifically adapted for displacement generators. Strassen's approach relies on recursive block-matrix partitioning and Schur complements,

Algorithm 7 Strassen-type Inversion for Displacement Generators (adapted from [1])

procedure StrassenInverseGenerators(G_A, H_A)
  if $n = 1$ **then** ▷ **Step 1: Base Case**
    return G_D H_D representing the inverse of the scalar value of A from G_A, H_A
  end if

  $G_a, H_a, \ldots, G_d, H_d \leftarrow$ SplitQuadrants(G_A, H_A) ▷ **Step 2: Split into Quadrants**
  ▷ **Step 3: Recursion#1**
  $G_e, H_e \leftarrow$ StrassenInverseGenerators(G_a, H_a) ▷ $e := a^{-1}$

  $G_{ce}, H_{ce} \leftarrow$ Compress(Mul(G_c, H_c, G_e, H_e)) ▷ **Step 4: Schur Complement**
  $G_{eb}, H_{eb} \leftarrow$ Compress(Mul(G_e, H_e, G_b, H_b))
  $G_{ceb}, H_{ceb} \leftarrow$ Compress(Mul(G_c, H_c, G_{eb}, H_{eb}) $\times -1$)
  $G_S, H_S \leftarrow$ Compress(Add($G_d, H_d, G_{ceb}, H_{ceb}$)) ▷ $S := d - ceb$
  ▷ **Step 5: Recursion#2**
  $G_t, H_t \leftarrow$ StrassenInverseGenerators(G_S, H_S) ▷ $t := S^{-1}$

  $G_{ebt}, H_{ebt} \leftarrow$ Compress(Mul(G_{eb}, H_{eb}, G_t, H_t)) ▷ **Step 6: Strassen Assembly**
  $G_{tce}, H_{tce} \leftarrow$ Compress(Mul(G_t, H_t, G_{ce}, H_{ce}))
  $G_{ebtce}, H_{ebtce} \leftarrow$ Compress(Mul($G_{ebt}, H_{ebt}, G_{ce}, H_{ce}$))

  $G_x, H_x \leftarrow$ Compress(Add($G_e, H_e, G_{ebtce}, H_{ebtce}$)) ▷ $x := e + ebtce$
  $G_y, H_y \leftarrow$ Negate(G_{ebt}, H_{ebt}) ▷ $y := -ebt$
  $G_z, H_z \leftarrow$ Negate(G_{tce}, H_{tce}) ▷ $z := -tce$

  $G_{Inv}, H_{Inv} \leftarrow$ PackQuadrants($G_x, H_x, G_y, H_y, G_z, H_z, G_t, H_t$) ▷ **Step 7: Pack**
  return Compress(G_{Inv}, H_{Inv})
end procedure

This algorithm is based on Algorithm 10.1 from [1, Algorithme 10.1]. Notice that we only operate on Φ_+ displacement and bypassing Φ_- generators.

Proposition 2. *The Strassen-type inversion algorithm maintains Φ_+ displacement generators throughout its recursive execution, without necessitating conversion to Φ_- generators.*

Proof. We proceed by induction on the size of the matrix $n = 2k$. Let $\Phi_+(X) = X - ZXZ^T$ be the displacement operator.

Base Case ($n = 1$): For a scalar matrix $A = [a]$, the operator Z is a 1×1 zero matrix. The displacement of A is $\Phi_+(A) = A - 0 \cdot A \cdot 0 = A$.

Inductive Step: Consider a matrix A of size $2k \times 2k$ represented by Φ_+ generators. Inversion relies on splitting the sub-blocks a, b, c, d which are represented by Φ_+ displacements, recursive calls to invert top-left block a ($e = a^{-1}$) and another for the Schur complement $t = S^{-1}$ and each outputs a Φ_+ generator. Multiplication and addition algorithms are taking and returning a generator of Φ_+ displacement. Each computation and the final assembly remain in Φ_+ displacement.

Because the base case returns the inverse to Φ_+ generators, and the arithmetics are on Φ_+ generator operations, by induction, we maintain Φ_+ displacement during the entire execution. □

4.2 Split and Pack quadrants from displacement generators

Any divide and conquer algorithm will divide the problem to create sub-problems to economise in complexity. This algorithm calls for a dense matrix A be split into four quadrants:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

Since we don't have direct access to the dense matrix A, any splitting and packing operation done will be on the generators G_A and H_A which represent the displacement matrix. When we attempt to evaluate the local displacement of a sub-block (for example, $\Phi_+(d)$), we cannot simply extract the rows from the global generators G_A and H_A. The global generators are corrupted by the boundary elements of a that shifted into b and c. We must correct these boundary crossings which we call bleeding.

4.2.1 Splitting

During the split phase, we extract the local generators for sub-blocks a, b, c, d. We begin by splitting the global generators $G_A, H_A \in \mathbb{R}^{n \times r}$ into top and bottom blocks of sizes $n_1 = ceil(n/2)$ and $n_2 = floor(n/2)$:

$$G = \begin{pmatrix} G_{\text{TOP}} \\ G_{\text{BOTTOM}} \end{pmatrix} \quad H = \begin{pmatrix} H_{\text{TOP}} \\ H_{\text{BOTTOM}} \end{pmatrix}$$

Block *a*: This block receives no shifted data from other quadrants. Its generators are the global top halves:

$$G_a = G_{\text{TOP}}, \quad H_a = H_{\text{BOTTOM}}$$

Block *b*: The last column of block a, denoted v_a, shifts right into the first column of block b. To correct this bleed, we must reconstruct v_a from the global generators. Naively evaluating the displacement sum for a boundary vector requires $O(n^2 r)$ operations. However, this evaluation is equivalent to computing the coefficients of a polynomial multiplication. By packing the columns of G and the reversed columns of H into polynomials, we extract v_a with polynomial multiplication in $O(r \cdot M(n))$ time. Once extracted, we shift v_a down using the operator Z_{n_1} and

append it to G_{TOP}. To apply this correction only to the first column of b, we append the vector $e_0^{(n_2)} = (1, 0, \ldots, 0)^T$ to H_{bot}:

$$G_b = \begin{pmatrix} G_{\text{TOP}} & Z_{n_1} v_a \end{pmatrix}, \quad H_b = \begin{pmatrix} H_{\text{BOTTOM}} & e_0^{(n_2)} \end{pmatrix}$$

Block *c*: Bleeding occurs to c from above via the last row of a, denoted r_a. Using the same polynomial operation, we recover r_a. Because H generators represent the transposed displacement components, we apply the shift Z_{n_1} to r_a and append it to H_{TOP}:

$$G_c = \begin{pmatrix} G_{\text{BOTTOM}} & e_0^{(n_2)} \end{pmatrix}, \quad H_c = \begin{pmatrix} H_{\text{TOP}} & Z_{n_1} r_a \end{pmatrix}$$

Block *d*: Block d is exposed to bleeding from all directions: the last column of c (v_c) shifting right, the last row of b (r_b) shifting down, and the exact bottom-right scalar of a (s_a) shifting diagonally down-right. We extract v_c and r_b using polynomial operation. Because s_a is a single cell, polynomial overhead is unnecessary; it is computed directly in (nr) time. Scaling the vector $e_0^{(n_2)}$ by s_a and addint to the shifted vectors:

$$G_d = \begin{pmatrix} G_{\text{BOTTOM}} & s_a e_0^{(n_2)} + Z_{n_2} v_c & e_0^{(n_2)} \end{pmatrix}, \quad H_d = \begin{pmatrix} H_{\text{BOTTOM}} & e_0^{(n_2)} & Z_{n_2} r_b \end{pmatrix}$$

4.2.2 Packing

The pack operation reverts the changes of the splitting phase. After computing the inverse sub-blocks (denoted as x, y, z, t), our goal is to assemble the global generators G_{INV} and H_{INV} for the full inverse matrix A^{-1}. If we were to concatenate directly the local generators, the result would assume that the boundaries between the sub-blocks are filled with zeros. Because the global operator Φ_+ shifts elements, we must correct the bleeding to restore data integrity. Just as we calculated boundaries during the split, we must extract the boundary vectors of the local inverse blocks. Since we are now operating on isolated sub-block generators, the coordinates for each reconstruction are local and starts from zero.

To apply these corrections, we construct a large set of global generators. We horizontally concatenate the local generators of x, y, z, t into their respective quadrants and append four specific boundary correction columns (c_1, c_2, c_3, c_4). We define these global assembled generators as block matrices:

$$G_{RES} = \begin{pmatrix} G_x & G_y & 0 & 0 & -Z_{n_1} v_x & 0 & 0 & 0 \\ 0 & 0 & G_z & G_t & 0 & -e_0^{(n_2)} & -(s_x e_0^{(n_2)} + Z_{n_2} v_z) & -e_0^{(n_2)} \end{pmatrix}$$

$$H_{RES} = \begin{pmatrix} H_x & 0 & H_z & 0 & 0 & Z_{n_1} r_x & 0 & 0 \\ 0 & H_y & 0 & H_t & e_0^{(n_2)} & 0 & e_0^{(n_2)} & Z_{n_2} r_y \end{pmatrix}$$

Notice that the boundary correction columns in G_{RES} carry negative signs. The reconstruction is going to add the values diagonally, to correct bleeding, we must substract the the corrections. To prove the correctness of these columns, we compute the global displacement reconstruction by multiplying $\Phi_+(A^{-1}) = G_{RES} H_{RES}^T$:

$$G_{RES} H_{RES}^T = \begin{pmatrix} G_x H_x^T & G_y H_y^T - Z_{n_1} v_x (e_0^{(n_2)})^T \\ G_z H_z^T - e_0^{(n_2)} (Z_{n_1} r_x)^T & G_t H_t^T - Z_{n_2} v_z (e_0^{(n_2)})^T - e_0^{(n_2)} (Z_{n_2} r_y)^T - s_x e_0^{(n_2)} (e_0^{(n_2)})^T \end{pmatrix}$$

The isolated terms ($G_x H_x^T, G_y H_y^T$, etc.) represent the isolated local displacements of the sub-matrices. The subtracted terms are the bleeds. For example, the term $-Z_{n_1} v_x (e_0^{(n_2)})^T$ subtracts the shifted right column of block x as it crosses into the first column of block y.

This structural reassembly exposes a vulnerability. Let $\alpha_x, \alpha_y, \alpha_z, \alpha_t$ be the displacement ranks of the sub-quadrants. The rank of our assembled global generator is:

$$\text{Rank}_{RES} = \alpha_x + \alpha_y + \alpha_z + \alpha_t + 4$$

Passing this uncompressed matrix up to the next recursion level would cause the displacement rank to explode, ruining the optimisations we aimed for at the first place. We call the compression seen in section 3 to keep the algorithm efficient.

4.3 Inversion Benchmarks

The performance benchmarks for the generator-based Strassen inversion show improvements over standard dense operations at larger scales:

n	Generators (ms)	Flint Dense (ms)
128	5.328	1.967
256	12.79	12.03
512	30.74	80.16
1024	74.50	554.8
2048	180.4	3975
4096	452.9	28300

Table 4: Average runtimes of inversion operations ($\alpha = 2$). Averages are computed over 100 iterations for $n \leq 512$, 20 iterations for $n \in \{1024, 2048\}$, and 10 iterations for $n = 4096$.

Method | α	$n = 512$ (ms)	$n = 1024$ (ms)	$n = 2048$ (ms)
Generators ($\alpha = 2$)	34.65	77.96	180.4
Generators ($\alpha = 4$)	61.12	144.8	340.2
Generators ($\alpha = 8$)	135.5	333.5	800.1
Generators ($\alpha = 16$)	363.8	931.5	2259.0
Generators ($\alpha = 32$)	1064.0	2829.0	7313.0
Flint Dense (Baseline)	80.16	554.8	3975.0

Table 5: Average runtimes of generator inversion scaling by rank over 10 runs, compared to constant dense inversion baseline.

We analyze the execution time as n grows. While the split and pack operate in $O(\alpha M(n))$. The recursive algorithm involves multiplying matrices with generators seen in section 2.2.2 and thus overall algorithm is bounded by $O(\alpha^2 M(n) \log n)$.

Taking $M(n) = O(n \log n)$, the theoretical complexity is $O(\alpha^2 n \log^2 n)$. For a constant rank, the expected scaling factors when increasing n from 128 to 1024 (8×) and from 1024 to 4096 (4×) are approximately 16.32 and 5.76 respectively. The observed scaling ratios are:

$$\frac{T(1024)}{T(128)} \approx 14.43, \quad \frac{T(4096)}{T(1024)} \approx 6.02$$

These observed ratios seem to align with the expectations. The slight deviation at smaller sizes (14.43 and 16.32) can be explained by the recursive overhead and the constant factors of

copying matrix elements into polynomials. Also the difference at larger sizes (6.02 and 5.76) can be explained as the data structure scales up, they begin to exceed the CPU's fast cache limits. Eventhough, as n progresses, complexity respects the $O(\alpha^2 M(n) \log n)$ bound. When comparing against the dense approach, FLINT follows an $O(n^3)$ curve to over 28 seconds at $n = 4096$. Inversion with generators completes the same operation in just over 450 milliseconds, giving us a $\sim 62\times$ speedup for a Toeplitz matrix. But this speedup will decrease as rank increases on a quasi-Toeplitz case, as the efficiency of storing a dense matrix decreases too. However, the results reveal an intersection point. For small matrices ($n \leq 256$), the recursive overhead and cost of splitting and packing the sub matrices make the algorithm slower than dense inversion. As seen in table 4, the crossover occurs between $n = 256$ and $n = 512$.

Table 5 highlights the algorithm's dependency to the displacement rank α. Since the overall complexity contains linear operations (splitting and packing in $O(\alpha)$) and quadratic operations (generator multiplication in $O(\alpha^2)$), we expect the α^2 term as α grows. Doubling the rank from 2 to 4 increased the runtime by roughly $1.85\times$, but doubling it from 16 to 32 increased the runtime by a factor of over $3.2\times$. Because the algorithm scales quadratically any doubling of the rank forces the algorithm to do 4 times as much work This confirms that to maintain high performance in iterative applications, periodic generator compression (as detailed in Section 3) is absolutely critical.

5 Conclusion and Perspectives

In this project, we successfully implemented arithmetic algorithms for quasi-Toeplitz matrices using displacement generators. However, while Strassen-type inversion is highly effective for large matrices with small ranks, the rank explosion problem and computational overhead limit its viability for smaller systems or highly iterative applications.

For future optimizations, alternative algorithmic structures could be preferred. As detailed by Victor Y. Pan [6, Chapter 5], the Morf-Bitmead-Anderson (MBA) algorithm achieves an even faster $O(n \log^2 n)$ complexity for Toeplitz-like matrices. However, as it is also a divide-and-conquer method, it suffers from similar calculations and generator compression. To bypass matrix splitting entirely, the Newton iterative version based on the Schulz iteration $X_{k+1} = X_k(2I - AX_k)$—is highly preferable [6, Chapters 6 and 7]. If the objective is to solve a linear system $Ax = b$, computing the full inverse A^{-1} introduces unnecessary computations. Instead, one can take direct advantage of the structured properties of A to solve the system more efficiently. Methods such as the Levinson-Durbin algorithm [4], avoid inversion and solve Toeplitz systems in $O(n^2)$ operations. Iterative Krylov subspace methods, the Preconditioned Conjugate Gradient (PCG) method [2] use FFT to compute matrix-vector products, achieving $O(n \log n)$ complexity per iteration. These iterative methods they rely on numerical approximation and convergence tolerances. For an exact deterministic algebraic solution, there is an alternative by resolving via M-Padé approximations[5], which achieves a deterministic $\tilde{O}(\alpha^{\omega-1} n)$ complexity.

References

[1] Alin Bostan, Frédéric Chyzak, Marc Giusti, Romain Lebreton, Grégoire Lecerf, Bruno Salvy, and Éric Schost. Algorithmes efficaces en calcul formel, August 2017. 686 pages. Édition 1.0. URL: https://hal.archives-ouvertes.fr/AECF/.

[2] Tony F. Chan. An optimal circulant preconditioner for toeplitz systems. *SIAM Journal on Scientific and Statistical Computing*, 9(4):766–771, 1988. arXiv:https://doi.org/10.1137/0909051, doi:10.1137/0909051.

[3] Ignat Domanov and Lieven De Lathauwer. Generic uniqueness of a structured matrix factorization and applications in blind source separation. *IEEE Journal of Selected Topics in Signal Processing*, 10(4):701–711, 2016. doi:10.1109/JSTSP.2016.2526971.

[4] James Durbin. The fitting of time-series models. *Revue de l'Institut International de Statistique*, pages 233–244, 1960.

[5] Sara Khichane and Vincent Neiger. Matrices with displacement structure: a deterministic approach for linear systems and nullspace bases, 2026. URL: https://arxiv.org/abs/2603.02425, arXiv:2603.02425.

[6] Victor Y. Pan. *Structured Matrices and Polynomials: Unified Superfast Algorithms*. Birkhäuser & Springer, 2001. URL: https://doi.org/10.1007/978-1-4612-0129-8.

[7] Janusz P. Papliński, Aleksandr Cariow, Paweł Strzelec, and Marta Makowska. Digital signal processing (dsp)-oriented reduced-complexity algorithms for calculating matrix–vector products with small-order toeplitz matrices. *Signals*, 5(3):417–437, 2024. URL: https://www.mdpi.com/2624-6120/5/3/21, doi:10.3390/signals5030021.

[8] The FLINT team. *FLINT: Fast Library for Number Theory*, 2025. Version 3.3.1.

[9] S. S. Zhou and J. B. Wang. Efficient quantum circuits for dense circulant and circulant like operators. *Royal Society Open Science*, 4(5):160906, 05 2017. arXiv:https://royalsocietypublishing.org/rsos/article-pdf/doi/10.1098/rsos.160906/509313/rsos.160906.pdf, doi:10.1098/rsos.160906.

Appendix

Implementations

Displacement

`Project/src/displacement_matrices/*`

gr_mat_displacement(gr_mat_t D, gr_mat_t A, disp_type_t type, gr_ctx_t ctx) Is the function taking as parameter the matrix A and returns into the matrix D, indicating destination, the displacement matrix of A. According to the type of displacement we pass via the parameter `type`, it calculates the implementations (2) or (3).

int gr_mat_G_H(gr_mat_t G, gr_mat_t H, gr_mat_t A, disp_type_t type, gr_ctx_t ctx)) Is the function that computes the generator matrices G and H^T for an input matrix A. Function performs an LU decomposition on the displacement matrix D. It then extracts the independent columns to form G and the corresponding rows to form H^T.

int gr_mat_reconstruct_A_v2(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx); Is the function that reconstructs the original matrix A from its generator matrices G and H.

Compression

`Project/src/compression/*`

int gr_mat_generator_compress(gr_mat_t G_d, gr_mat_t H_d, gr_ctx_t ctx); Is our function that implements the algorithms talked in chapter 3.

Inversion

`Project/src/inverse_topelitz/*`

gr_mat_inverse_toeplitz(gr_mat_t G_D, gr_mat_t H_D, gr_mat_t G_A, gr_mat_t H_A, gr_ctx_t ctx) Implements the generator inversion talked in section 4. Same directory also houses the auxilary functions for packing and unpacking.

Utility

`Project/src/utility/*`

int gr_mat_apply_Z(gr_mat_t Res, gr_mat_t M, gr_ctx_t ctx); Function that we implement to apply the downshift matrix Z to a matrix M and store the result in Res.

int gr_mat_apply_Zt(gr_mat_t Res, gr_mat_t M, gr_ctx_t ctx); Function that we implement to apply the transpose of the downshift matrix Z^T to a matrix M and store the result in Res.